

The Tinkerers



The Tinkerers | 2026

Server-Side WebAssembly in Practice



Storm van Loon, Skaara Poncin, Gabriel Uwaila

1. Agenda

01

WebAssembly & WASI
Basics

02

Developer experience:
runtimes, tooling,
components

03

HTTP servers & AI with
WASM

04

WASM in the cloud:
containers, hosts, edge,
Kubernetes

05

Hype decline and future
directions (TEE, agents,
edge, WASI 0.3)

06

Q&A

1. WebAssembly Basics

(wasm = WebAssembly)

01

WebAssembly & WASI
Basics

02

Developer experience:
runtimes, tooling,
components

03

HTTP servers & AI with
WASM

04

WASM in the cloud:
containers, hosts, edge,
Kubernetes

05

Hype decline and future
directions (TEE, agents,
edge, WASI 0.3)

06

Q&A

What is WebAssembly (on the client side) ?

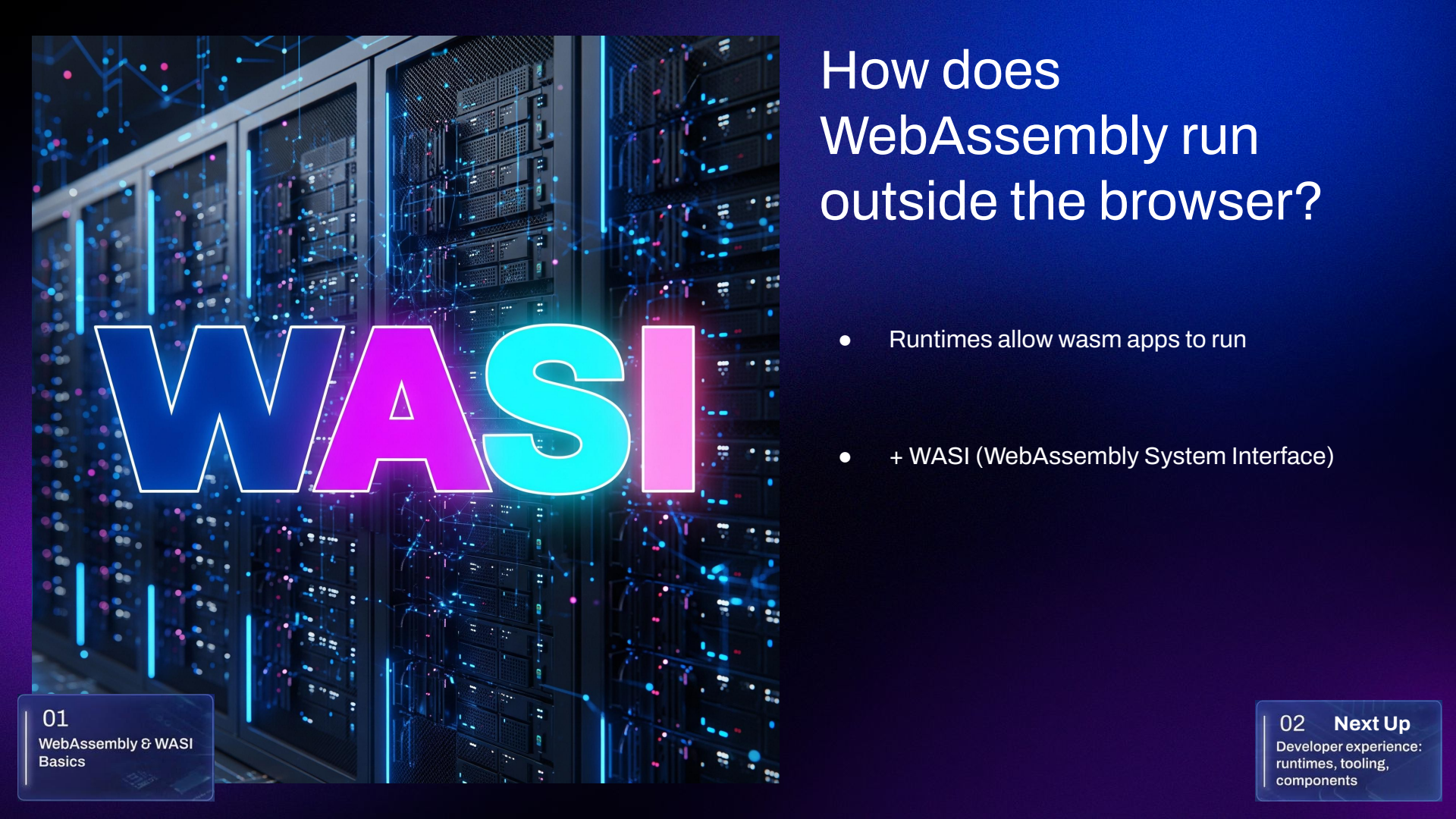
- Designed to fix JavaScript limitations.
- Compact binary format for very efficient execution.
- Performance-intensive Applications

A large, bold, white 'WA' logo is centered on a solid blue rectangular background. The letters are thick and blocky, with a slight shadow effect. The blue background is part of a larger slide design that includes a dark blue header and a purple gradient footer.

How does it differ from Server Side Wasm ?

- Compact binary format
- Secure, Sandboxed environment
- Portable
- More efficient than standard linux containers





How does WebAssembly run outside the browser?

- Runtimes allow wasm apps to run
- + WASI (WebAssembly System Interface)

01

WebAssembly & WASI
Basics

02 **Next Up**

Developer experience:
runtimes, tooling,
components

2. Developer experience

(the promises... 🤖)

01

WebAssembly & WASI
Basics

02

Developer experience:
runtimes, tooling,
components

03

HTTP servers & AI with
WASM

04

WASM in the cloud:
containers, hosts, edge,
Kubernetes

05

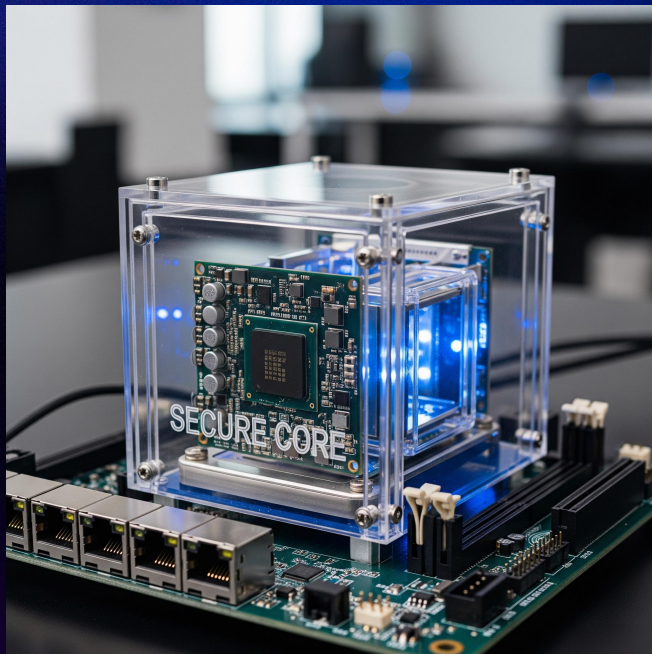
Hype decline and future
directions (TEE, agents,
edge, WASI 0.3)

06

Q&A

WASM is Security First... Right?

- Sandboxed, memory-safe modules.
- Explicit imports.
- Security downside = binary difficult to analyze.
- .wat solves this problem



02

Developer experience:
runtimes, tooling,
components

The Cross Language Promise

- Compile once, run everywhere
- Prevent Unauthorized library access
- No need to rebuild the host application.
- Create safe plugins.

02

Developer experience:
runtimes, tooling,
components

Cross Language Interoperability in Practice

- Rust has great support
- C/C++ has decent support
- Go has mediocre support
- Python has very limited support

02

Developer experience:
runtimes, tooling,
components



Demo one : Running a compiled .wasm file in Java

02

Developer experience:
runtimes, tooling,
components

03 **Next Up**

HTTP servers & AI with
WASM

3a. HTTP Servers

01

WebAssembly & WASI
Basics

02

Developer experience:
runtimes, tooling,
components

03

HTTP servers & AI with
WASM

04

WASM in the cloud:
containers, hosts, edge,
Kubernetes

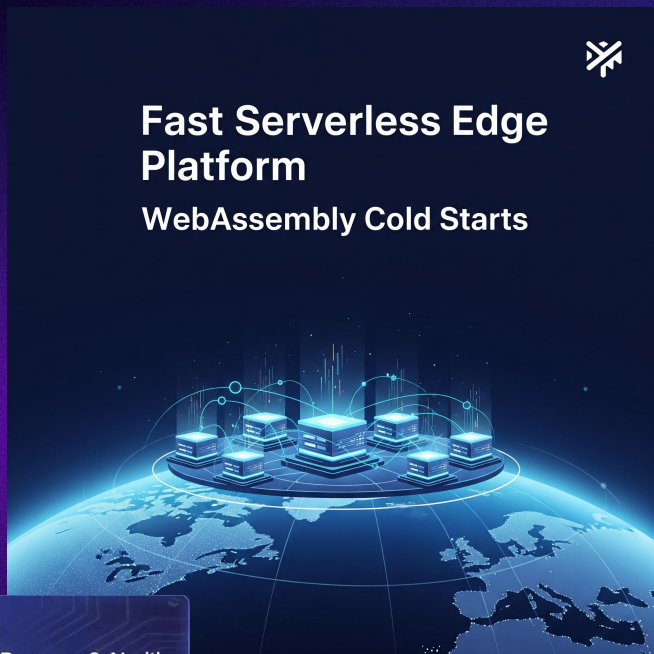
05

Hype decline and future
directions (TEE, agents,
edge, WASI 0.3)

06

Q&A

Why Wasm for HTTP?



- Portability allows the same file to run on any host.
- HTTP applications are sandboxed with good security.
- Minimal capabilities are required through the WASI interface.
- Extremely fast cold starts benefit serverless and edge platforms.

03

HTTP servers & AI with
WASM

3b. AI with WASM

03

HTTP servers & AI with
WASM

Is WASM helpful in the AI goldrush?

- AI inference logic can be compiled into WebAssembly. (FaaS)
- Execution of Wasm AI models is very fast, often in milliseconds.
- Spin simplify the deployment and execution.



DEMO 2 Moving heavy AI logic + API to WASM.

03

HTTP servers & AI with
WASM

4. WASM in the sky

(In the cloud)

01

WebAssembly & WASI
Basics

02

Developer experience:
runtimes, tooling,
components

03

HTTP servers & AI with
WASM

04

WASM in the cloud:
containers, hosts, edge,
Kubernetes

05

Hype decline and future
directions (TEE, agents,
edge, WASI 0.3)

06

Q&A

Wasm vs. Containers

Wasm



- Lightweight
- Fast Startup
- Language Agnostic
- Secure Sandbox
- Portable
- Small Footprint

Containers



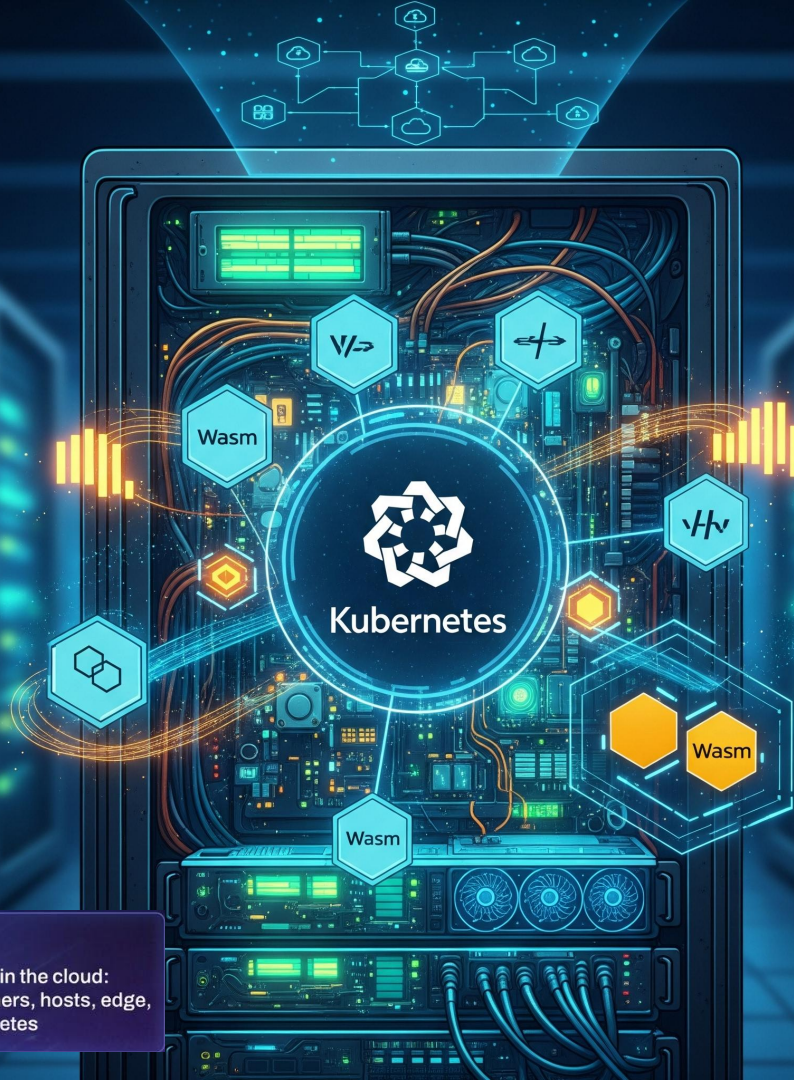
- Isolated Environment
- Operating System Virtualization
- Package Dependencies
- Mature Ecosystem
- Resource-Heavy

Wasm and Kubernetes

- Direct Kubernetes with pure WebAssembly is difficult
- Solutions are either docker-dependent or not maintained
- SpinKube and WasmCloud.

04

WASM in the cloud:
containers, hosts, edge,
Kubernetes



DEMO 3 : WASM Cloud

04

WASM in the cloud:
containers, hosts, edge,
Kubernetes

Wasm at the Edge

01 — CDN platforms execute Wasm code close to users.

02 — Wasm UDFs extend database engines with custom functions.

03 — Fast startup times benefit serverless and edge platforms.

04 — Microcontrollers use runtimes like WAMR and wasm3.



04

WASM in the cloud:
containers, hosts, edge,
Kubernetes

5. Hype decline & future

01

WebAssembly & WASI
Basics

02

Developer experience:
runtimes, tooling,
components

03

HTTP servers & AI with
WASM

04

WASM in the cloud:
containers, hosts, edge,
Kubernetes

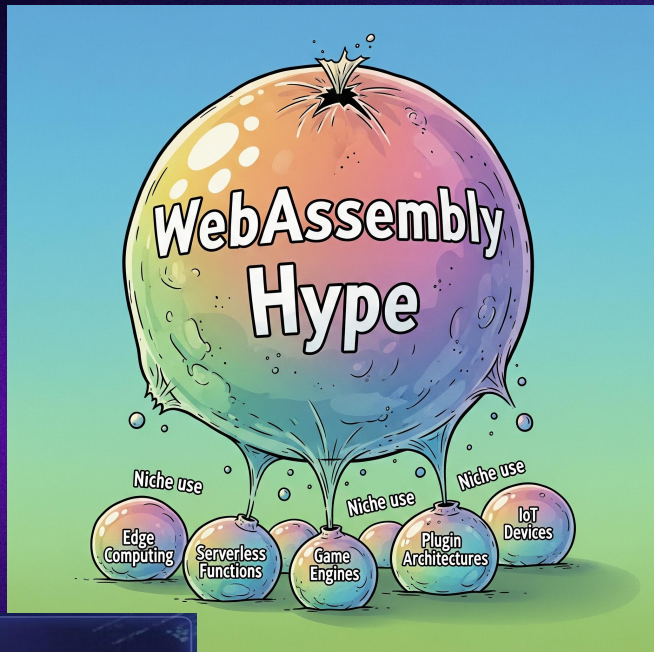
05

Hype decline and future
directions (TEE, agents,
edge, WASI 0.3)

06

Q&A

Wasm Hype vs. Reality



- Early hype suggested Wasm would replace JVM and containers
- Reality shows significant gaps in documentation and tooling
- Market has cooled down significantly for server-side wasm
- wasm's main role is in niches and with big companies

05

Hype decline and future directions (TEE, agents, edge, WASI 0.3)

Wasm's Current Wins

01 Cross-language interoperability creates safe plugin architectures.

03 Fast cold starts benefit serverless and edge workloads.

02 It is a key solution for confidential computing and AI agents.

04 Wasm is useful for rewriting microservices as FaaS.



05

Hype decline and future directions (TEE, agents, edge, WASI 0.3)

The Future of wasm?

01 — Wasm and TEEs enable secure confidential computing.

02 — WASI 0.3 features will enable concurrent I/O and streams.

03 — AI agents use sandboxing to limit prompt injection risks.

04 — Edge WAF logic enhances security with reduced latency.

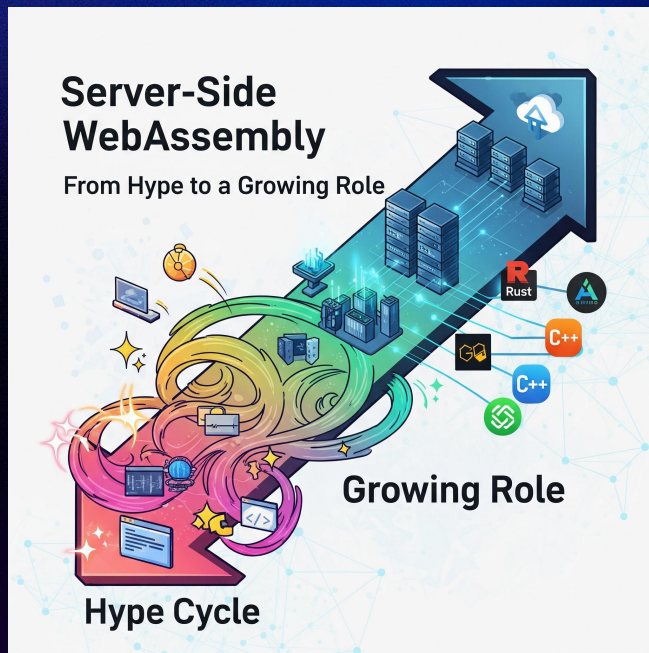


05

Hype decline and future directions (TEE, agents, edge, WASI 0.3)

Conclusion Summary and Future

- From hype to a growing role.
- Strong fit for edge/FaaS, plugins, and AI agents.
- Documentation, adoption, and implementation still severely lack.



The End

- Questions about our presentation are welcome
- Visit the website!

